

Архитектура и основные классы системы

=====

Общая структура системы

Архитектура игры построена по принципу разделения ответственности между классами. Каждый класс отвечает только за свою часть логики: персонажи, бой, предметы, локации, диалоги, торговля и общий игровой процесс.

Основу системы составляет базовый класс `Character`, от которого наследуются `Hero` и `Enemy`.

Для реализации предметов используется базовый класс `Item`, от которого наследуются классы `Weapon` и `UsableItem`.

Игровой процесс объединяется классом `Game`, который взаимодействует с остальными компонентами системы.

Для хранения и загрузки игровых данных из таблиц используется класс `GameDatabase`.

Такой подход позволяет:

- разделить логику игры на независимые части;
- упростить расширение системы;
- задавать игровые параметры через таблицы;
- сохранить соответствие ГДД и реализуемой механике.

Схема связей между классами

```
Character
├── Hero
└── Enemy

Item
├── Weapon
└── UsableItem
```

```
Game
├── BattleSystem
├── DialogueSystem
├── Castle
├── Merchant
└── GameDatabase

Castle
└── Room

Merchant
└── Shop

Shop
└── Item

DialogueSystem
└── DialogueChoice

GameDatabase
├── WeaponData
├── UsableItemData
├── EnemyData
├── RoomData
└── DialogueChoiceData
```

Класс Character

=====

Краткое описание

Класс Character является базовым классом для всех персонажей игры.

Хранит характеристики, состояние частей тела, реализует получение урона с учётом случайного шанса попадания и поддерживает систему временных эффектов.

Отвечает за

характеристики персонажа;

состояние частей тела;

расчёт попадания;

расчёт урона;

бафы и дебафы;

состояние персонажа.

Не отвечает за

проведение боя;

управление игрой;

диалоги;

торговлю.

Структура класса

```
#include <string>
#include <vector>

struct BodyParts {
    int head;    // здоровье головы
    int body;    // здоровье тела
    int arms;    // здоровье рук
    int legs;    // здоровье ног
};

struct ActiveEffect {
    std::string stat; // изменяемая характеристика
    int value;        // величина изменения
    int turns_left;   // оставшееся количество ходов
}
```

```
};

class Character {
public:
    // Конструктор персонажа
    Character(std::string name, int atk, int def, int agi, BodyParts parts);

    // Получить урон по голове
    bool TakeDamageToHead(int incoming_attack);

    // Получить урон по телу
    bool TakeDamageToBody(int incoming_attack);

    // Получить урон по рукам
    bool TakeDamageToArms(int incoming_attack);

    // Получить урон по ногам
    bool TakeDamageToLegs(int incoming_attack);

    // Лечение головы
    void HealHead(int value);

    // Лечение тела
    void HealBody(int value);

    // Лечение рук
    void HealArms(int value);

    // Лечение ног
    void HealLegs(int value);

    // Лечение всех частей тела
    void HealAll(int value);

    // Проверка попадания
    bool RollHit(int base_chance) const;

    // Обновить состояние персонажа
    void UpdateAliveState();

    // Добавить временный эффект
    void AddEffect(const ActiveEffect& effect);
};
```

```

// Обновить временные эффекты
void UpdateEffects();

// Получить атаку с учетом эффектов
int GetAttack() const;

// Получить защиту с учетом эффектов
int GetDefense() const;

// Получить ловкость с учетом эффектов
int GetAgility() const;

// Проверить, жив ли персонаж
bool IsAlive() const;

protected:
    std::string name; // имя персонажа

    int attack; // базовая атака
    int defense; // базовая защита
    int agility; // базовая ловкость

    BodyParts parts; // текущее состояние частей тела

    bool alive; // жив ли персонаж
    bool arms_broken; // сломаны ли руки
    bool legs_broken; // сломаны ли ноги

    std::vector<ActiveEffect> active_effects_; // активные временные
эффекты
};

```

=====

Проверка смерти

```

bool Character::RollHit(int base_chance) const {
    // Расчёт итогового шанса попадания с учётом ловкости цели
    int final_chance = base_chance - GetAgility();

    // Ограничение минимального шанса попадания
    if (final_chance < 10) {
        final_chance = 10;
    }

    // Генерация случайного числа от 1 до 100
    int rand_value = rand() % 100 + 1;

    // Проверка попадания
    return rand_value < final_chance;
}

```

=====

Примеры методов получения урона

=====

Проверка попадания

```

bool Character::rollHit(int baseChance) const {
    int finalChance = baseChance - getAgility();

    if (finalChance < 10)
        finalChance = 10;

    int randValue = rand() % 100 + 1;

    return randValue < finalChance;
}

```

Голова

```
bool Character::TakeDamageToHead(int incoming_attack) {
    if (!rollHit(20)) {
        return false;
    }

    int damage = (incoming_attack * 2) - getDefense();

    if (damage < 1) {
        damage = 1;
    }

    parts.head -= damage;

    if (parts.head < 0) {
        parts.head = 0;
    }

    updateAliveState();

    return true;
}
```

Тело

```
bool Character::takeDamageToBody(int incomingAttack) {
    if (!rollHit(80))
        return false;

    int damage = incomingAttack - getDefense();

    if (damage < 1)
        damage = 1;
```

```
parts.body -= damage;

if (parts.body < 0)
    parts.body = 0;

updateAliveState();

return true;
}
```

Руки

```
bool Character::takeDamageToArms(int incomingAttack) {
    if (!rollHit(60))
        return false;

    int damage = (incomingAttack * 0.9) - getDefense();

    if (damage < 1)
        damage = 1;

    parts.arms -= damage;

    if (parts.arms <= 0) {
        parts.arms = 0;
        armsBroken = true;
    }

    updateAliveState();
    return true;
}
```

Ноги

```
bool Character::takeDamageToLegs(int incomingAttack) {
    if (!rollHit(65))
        return false;

    int damage = (incomingAttack * 0.8) - getDefense();

    if (damage < 1)
        damage = 1;

    parts.legs -= damage;

    if (parts.legs <= 0) {
        parts.legs = 0;
        legsBroken = true;
    }

    updateAliveState();
    return true;
}
```

Реализация эффектов

=====

Добавление эффекта

```
void Character::addEffect(const ActiveEffect& effect) {
    activeEffects.push_back(effect);
}
```

Чек эффекта

```
void Character::updateEffects() {
    for (int i = 0; i < activeEffects.size(); ) {
        activeEffects[i].turnsLeft--;

        if (activeEffects[i].turnsLeft <= 0) {
```

```
        activeEffects.erase(activeEffects.begin() + i);
    } else {
        i++;
    }
}
```

Изменение характеристик

Атаки

```
int Character::getAttack() const {
    int total = attack;

    for (const auto& e : activeEffects)
        if (e.stat == "attack")
            total += e.value;

    if (armsBroken)
        total -= 3;

    if (total < 1)
        total = 1;

    return total;
}
```

Защиты

```
int Character::getDefense() const {
    int total = defense;

    for (const auto& e : activeEffects)
        if (e.stat == "defense")
            total += e.value;

    return total;
}
```

Ловкости

```
int Character::getAgility() const {
    int total = agility;

    for (const auto& e : activeEffects)
        if (e.stat == "agility")
            total += e.value;

    if (legsBroken)
        total -= 3;

    if (total < 0)
        total = 0;

    return total;
}
```

Примеры методов лечения

=====

Лечение головы

```
void Character::healHead(int value) {
    parts.head += value;

    if (parts.head > 100)
        parts.head = 100;

    updateAliveState();
}
```

Лечение тела

```
void Character::healBody(int value) {
    parts.body += value;

    if (parts.body > 100)
```

```
        parts.body = 100;

        updateAliveState();
    }
}
```

Лечение рук

```
void Character::healArms(int value) {
    parts.arms += value;

    if (parts.arms > 100)
        parts.arms = 100;

    if (parts.arms > 0)
        armsBroken = false;
}
```

Лечение ног

```
void Character::healLegs(int value) {
    parts.legs += value;

    if (parts.legs > 100)
        parts.legs = 100;

    if (parts.legs > 0)
        legsBroken = false;
}
```

Лечение всех частей тела

```
void Character::healAll(int value) {
    healHead(value);
    healBody(value);
    healArms(value);
    healLegs(value);
}
```

```
}
```

Класс Hero

=====

Краткое описание

Класс `Hero` описывает главного героя игры и расширяет класс `Character`.

Отвечает за

- атаки по частям тела;
- действия героя в бою;
- связь с инвентарём героя.

Не отвечает за

- проведение боя целиком;
- управление игрой;
- диалоги;
- торговлю.

Структура класса

```
class Inventory;

class Hero : public Character {
public:
    // Конструктор героя
    Hero(std::string name, int atk, int def, int agi, BodyParts parts);

    // Атака по голове
    int AttackHead() const;
```

```
// Атака по телу
int AttackBody() const;

// Атака по рукам
int AttackArms() const;

// Атака по ногам
int AttackLegs() const;

// Установить инвентарь героя
void SetInventory(Inventory* inv);

// Получить инвентарь героя
Inventory* GetInventory() const;

private:
    Inventory* inventory_; // инвентарь героя
};
```

Класс Enemy

=====

Краткое описание

Класс **Enemy** описывает врагов игры и наследуется от класса **Character**.

Отвечает за

- характеристики врага;
- выбор зоны атаки;
- действия врага в бою.

Не отвечает за

- управление игрой;
- диалоги;
- торговлю.

Структура класса

```
class Enemy : public Character {
public:
    Enemy(std::string name, int atk, int def, int agi, BodyParts parts); //
    создание врага

    int chooseAttackPart() const; // случайный выбор зоны атаки: 0 -
    голова, 1 - тело, 2 - руки, 3 - ноги
};
```

Логика атаки врага

Противник тоже атакует часть тела случайно.
Выбор зоны атаки происходит случайным образом.

Пример:

```
int Enemy::chooseAttackPart() const {
    return rand() % 4; // случайный выбор части тела
}
```

Класс BattleSystem

=====

Краткое описание

Класс `BattleSystem` реализует пошаговую боевую систему игры.
Учитывает случайный выбор зоны атаки врагом и атаки героя по выбранной части тела.

Отвечает за

- запуск боя;
- порядок ходов;
- обработку атак по зонам;
- определение победы или поражения.

Не отвечает за

- управление игрой;
- диалоги;
- торговлю;
- хранение характеристик персонажей.

Структура класса

```
class BattleSystem {
public:
    // Запуск боя, возвращает результат (победа/поражение)
    bool StartBattle(Hero& hero, Enemy& enemy);

private:
    // Ход игрока
    void PlayerTurn(Hero& hero, Enemy& enemy);

    // Ход врага
    void EnemyTurn(Hero& hero, Enemy& enemy);
};
```

В МЕТОДАХ ПОСЛЕ КАЖДОГО ХОДА ОБНОВЛЯЕМ ЭФФЕКТЫ

```
hero.updateEffects();
```



```
enemy.updateEffects();
```

Логика боя

Бой проходит пошагово:

- игрок выбирает часть тела;
 - враг выбирает случайно;
 - применяется урон;
 - после каждого хода обновляются эффекты персонажей;
 - бой продолжается до смерти одного из участников.
-

Класс Item

=====

Краткое описание

Класс `Item` является базовым классом для всех предметов игры.

Отвечает за

- общие свойства предметов;
- идентификатор предмета;
- название предмета;
- тип предмета.

Не отвечает за

- проведение боя;
- управление игрой;

- хранение инвентаря.

Структура класса

```
class Item {
protected:
    int id;           // идентификатор предмета
    std::string name; // название предмета
    std::string type; // тип предмета

public:
    Item(int id, std::string name, std::string type); // создание предмета
    virtual ~Item() = default;
};
```

Класс Weapon

=====

Краткое описание

Класс **Weapon** описывает оружие, которое наносит урон врагу.

Отвечает за

- хранение параметров оружия;
- урон оружия;
- точность оружия;
- применение оружия к врагу.

Не отвечает за

- хранение инвентаря;

- изменение характеристик героя;
- управление игрой.

Структура класса

```
struct WeaponData {
    int id;           // идентификатор оружия
    std::string name; // название оружия
    std::string type; // тип оружия
    int damage;       // урон оружия
    int accuracy;     // точность оружия
};

class Weapon : public Item {
private:
    int damage;       // урон оружия
    int accuracy;     // точность оружия

public:
    Weapon(const WeaponData& data); // создание оружия из данных

    int getDamage() const;          // получить урон
    int getAccuracy() const;        // получить точность
    void useOnEnemy(Enemy& enemy, int bodyPart); // применить оружие к врагу
};
```

Класс UsableItem

=====

Краткое описание

Класс `UsableItem` описывает используемые предметы, которые изменяют характеристики героя.

Отвечает за

-
- создание эффекта;
 - хранение эффекта предмета;
 - применение используемого предмета.
 - временное изменение характеристик героя;

Не отвечает за

- хранение инвентаря;
- нанесение прямого урона врагу;
- управление игрой.

Структура класса

```
struct UsableItemData {
    int id;           // идентификатор предмета
    std::string name; // название предмета
    std::string type; // тип предмета
    int value;        // сила эффекта
};

class UsableItem : public Item {
private:
    int value;

public:
    UsableItem(const UsableItemData& data);

    void useOnHero(Hero& hero);
};
```

Пример метода использования предметов

```
void UsableItem::useOnHero(Hero& hero) {
    if (type == "medicine_head")
```

```
        hero.healHead(value);

    if (type == "medicine_body")
        hero.healBody(value);

    if (type == "medicine_arms")
        hero.healArms(value);

    if (type == "medicine_legs")
        hero.healLegs(value);

    if (type == "medicine_all")
        hero.healAll(value);

    if (type == "smoke_bomb") {
        ActiveEffect effect;
        effect.stat = "agility";
        effect.value = value;
        effect.turnsLeft = 3;
        hero.addEffect(effect);
    }

    if (type == "attack_boost") {
        ActiveEffect effect;
        effect.stat = "attack";
        effect.value = value;
        effect.turnsLeft = 3;
        hero.addEffect(effect);
    }

    if (type == "defense_boost") {
        ActiveEffect effect;
        effect.stat = "defense";
        effect.value = value;
        effect.turnsLeft = 3;
        hero.addEffect(effect);
    }
}
```

Класс Inventory

=====

Краткое описание

Класс `Inventory` хранит предметы героя и позволяет использовать выбранный предмет.

Отвечает за

- хранение предметов;
- добавление предметов;
- удаление предметов;
- использование предметов.

Не отвечает за

- проведение боя;
- управление игрой;
- сюжет.

Структура класса

```
void Inventory::UseItem(int index, Hero& hero, Enemy* enemy, int body_part)
{
    // Получение предмета из инвентаря
    auto item = items_[index];

    // Если предмет является оружием
    if (auto weapon = std::dynamic_pointer_cast<Weapon>(item)) {
        // Проверка, есть ли цель для атаки
        if (enemy != nullptr) {
            // Применение оружия к врагу по выбранной части тела
            weapon->UseOnEnemy(*enemy, body_part);
        }
    }

    // Если предмет является используемым (лечебным или бафом)
```

```
if (auto usable = std::dynamic_pointer_cast<UsableItem>(item)) {  
    // Применение эффекта предмета к герою  
    usable->UseOnHero(hero);  
}  
}
```

Логика инвентаря

Инвентарь хранит список предметов.

При использовании предмета:

- если предмет является оружием, вызывается атака по врагу;
- если предмет является используемым предметом, применяется эффект к герою.

Класс Room

=====

Краткое описание

Класс `Room` описывает одну комнату замка.

Отвечает за

- хранение содержимого комнаты;
- наличие врага;
- наличие предмета;
- наличие подсказки.

Не отвечает за

- проведение боя;

- управление игрой;
- диалоги.

Структура класса

```
class Room {  
private:  
    int enemyId;    // идентификатор врага в комнате  
    int itemId;     // идентификатор предмета в комнате  
};
```

Класс Castle

=====

Краткое описание

Класс `Castle` описывает замок как набор комнат.

Отвечает за

- хранение комнат;
- прохождение замка;
- состояние замка.

Не отвечает за

- боевую систему;
- диалоги;
- торговлю.

Структура класса

```
class Castle {
public:
    // Конструктор
    explicit Castle(int id);

    // Добавить комнату в замок
    void AddRoom(const Room& room);

    // Проверить, пройден ли замок
    bool IsCleared() const;

private:
    int id; // идентификатор замка

    std::vector<Room> rooms; // список комнат замка

    bool cleared; // пройден ли замок
};
```

Класс DialogueChoice

=====

Краткое описание

Класс `DialogueChoice` описывает один вариант ответа в диалоге.

Отвечает за

- текст варианта ответа;
- переход к следующему шагу;

Не отвечает за

- отображение всего диалога;
- проведение боя;
- управление игрой.

Структура класса

```
class DialogueChoice {
private:
    std::string text;      // текст варианта ответа
    int nextStep;         // следующий этап диалога

public:
    DialogueChoice(std::string text, int nextStep); // создание варианта
    ответа

    std::string getText() const; // получить текст варианта
    int getNextStep() const;     // получить следующий шаг
};
```

Класс DialogueSystem

=====

Краткое описание

Класс `DialogueSystem` реализует систему диалогов.

Отвечает за

- запуск диалога;
- вывод текста;
- обработку выбора;
- влияние выбора на дальнейшие события.

Не отвечает за

- проведение боя;
- торговлю;
- управление всей игрой.

Структура класса

```
class DialogueSystem {  
public:  
    void startDialogue(); // запуск диалога  
};
```

Класс Shop

=====

Краткое описание

Класс **Shop** описывает магазин предметов.

Отвечает за

- список товаров;
- отображение ассортимента;
- продажу предметов.

Не отвечает за

- проведение боя;
- игровой процесс;

- диалоги.

Структура класса

```
class Shop {
private:
    std::vector<std::shared_ptr<Item>> items; // список товаров магазина

public:
    void showItems() const; // показать товары
    void buyItem(int index, Inventory& inventory); // купить предмет и
добавить в инвентарь
};
```

Класс Merchant

=====

Краткое описание

Класс **Merchant** описывает странствующего торговца.

Отвечает за

- встречу с игроком;
- открытие торговли;
- связь игрока с магазином.

Не отвечает за

- проведение боя;
- управление игрой;
- прохождение замков.

Структура класса

```
class Merchant {  
private:  
    std::string name; // имя торговца  
    Shop shop;        // магазин торговца  
  
public:  
    Merchant(std::string name); // создание торговца  
  
    void trade(Hero& hero);      // начать торговлю с героем  
};
```

Класс GameDatabase

=====

Краткое описание

Класс `GameDatabase` отвечает за загрузку игровых данных из внешних таблиц и хранение этих данных в памяти.

Отвечает за

загрузку данных оружия;

загрузку данных используемых предметов;

загрузку данных врагов;

загрузку данных комнат;

загрузку данных диалогов;

хранение всех загруженных данных.

Не отвечает за

проведение боя;

поведение объектов в игре;

управление игровым процессом.

Структура класса

```
struct WeaponData {
    int id;           // идентификатор оружия
    std::string name; // название оружия
    std::string type; // тип оружия (kunai, bomb и т.д.)
    int damage;       // урон оружия
    int accuracy;     // точность оружия
};

struct UsableItemData {
    int id;           // идентификатор предмета
    std::string name; // название предмета
    std::string type; // тип (medicine, smoke_bomb и т.д.)
    int value;        // сила эффекта
};

struct EnemyData {
    int id;           // идентификатор врага
    std::string name; // имя врага
    int attack;       // атака врага
    int defense;      // защита врага
    int agility;      // ловкость врага
    BodyParts parts;  // части тела врага
};

struct RoomData {
    int id;           // идентификатор комнаты
    int castleId;     // к какому замку относится
    int enemyId;      // идентификатор врага
    int itemId;       // идентификатор предмета
};
```

```

struct DialogueChoiceData {
    int id;           // идентификатор варианта
    std::string text; // текст варианта ответа
    int nextStep;     // следующий шаг
};

struct CastleData {
    int id;           // идентификатор замка
    std::string name; // идентификатор замка

};

class GameDatabase {
private:
    std::vector<WeaponData> weapons;           // данные оружия
    std::vector<UsableItemData> usableItems;   // данные используемых
предметов
    std::vector<CastleData> castles;
    std::vector<EnemyData> enemies;           // данные врагов
    std::vector<RoomData> rooms;              // данные комнат
    std::vector<DialogueChoiceData> dialogues; // данные диалогов

public:
    void loadWeapons();           // загрузить оружие
    void loadUsableItems();       // загрузить используемые предметы
    void loadEnemies();           // загрузить врагов
    void loadRooms();             // загрузить комнаты
    void loadDialogues();         // загрузить диалоги
};

```

Класс Game

=====

Краткое описание

Класс `Game` является главным управляющим классом игры.

Отвечает за

- запуск игры;
- общий игровой цикл;
- переходы между этапами;
- взаимодействие между игровыми системами.

Не отвечает за

- расчёт урона;
- хранение игровых данных;
- отдельную логику диалогов.

Структура класса

```
class Game {  
private:  
    GameDatabase database;           // база игровых данных  
  
public:  
    void run();                      // запуск игры  
  
private:  
    void startGame();               // начало игры  
    void gameLoop();               // основной игровой цикл  
    void endGame();                 // завершение игры  
};
```
